Learning Graph Construction for Approximate Nearest Neighbor Search using HNSW

Divisht (24110112), Aditya Kumar (22110015), Yash Patel (21110243), Harshita Singh (22110299)

Divisht
IIT Gandhinagar
India
24110112@iitgn.ac.in

Aditya Kumar
IIT Gandhinagar
India
adityakumar@iitgn.ac.in

Yash Patel
IIT Gandhinagar
India
patelyash@iitgn.ac.in

Harshita Singh
IIT Gandhinagar
India
harshita.singh@iitgn.ac.in

Abstract

Approximate Nearest Neighbor (ANN) search plays a crucial role in modern machine learning systems such as recommendation engines, semantic search, and vector databases. Hierarchical Navigable Small Worlds (HNSW) is one of the most efficient structures for ANN but relies heavily on manually tuned parameters and heuristic graph construction. In this project, we propose a data-driven approach to learn optimal graph construction strategies for HNSW using Bayesian optimization. By optimizing parameters such as M and $efConstruction$, we aim to improve recall, latency, and overall efficiency compared to standard configurations.

Keywords

Approximate Nearest Neighbor, HNSW, Graph Optimization, Bayesian Optimization, Vector Search

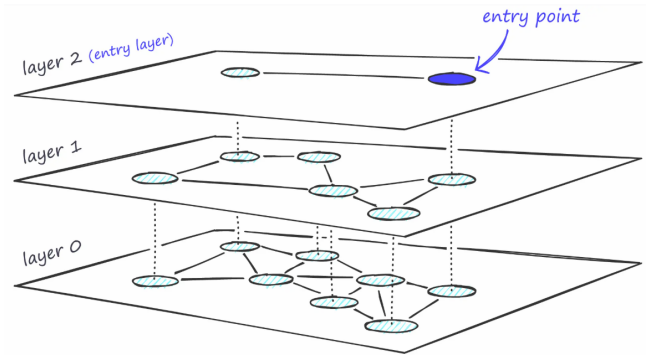

Figure 1: Hierarchical Navigable Small World (HNSW) structure

1 Introduction

Approximate Nearest Neighbor (ANN) search is a fundamental problem in machine learning systems such as recommendation engines, semantic search, and vector databases. One of the most efficient data structures for ANN is Hierarchical Navigable Small Worlds (HNSW), which constructs a multi-layer graph to enable fast search with logarithmic complexity [1].

However, HNSW relies on manually tuned parameters (e.g., M, $efConstruction$, $efSearch$) and heuristic graph-building strategies. Recent practical implementations and tutorials [3] highlight the importance of parameter tuning for performance.

As shown in Figure 1, HNSW enables efficient navigation through hierarchical graph layers.

2 Problem Statement

Current HNSW implementations rely on manually tuned parameters and heuristic-based graph construction. These configurations may not be optimal for different datasets. The objective of this project is to design a data-driven method to automatically learn optimal graph construction parameters to improve performance metrics such as recall, latency, and memory efficiency.

3 Implementation Plan

The implementation will follow a structured approach:

- **Dataset Preparation:** Prepare vector datasets and compute exact k-NN ground truth.
- **Parameter Tuning:** Expose key parameters:
  - M (maximum connections)
  - $efConstruction$ (insertion search depth)
- **Optimization:** Use Bayesian optimization (Optuna) to iteratively:
  - Generate parameter combinations
  - Construct HNSW graphs
  - Evaluate recall and latency
- **Validation:** Evaluate learned parameters on unseen datasets.

4 Tech Stack

- FAISS
- NumPy
- Scikit-learn
- hnswlib
- Optuna / Ray Tune

5 Evaluation and Analysis

We will benchmark the learned HNSW configuration against default settings using:

- Recall@k
- Latency (queries per second)

- Memory usage
- Indexing speed

The goal is to demonstrate measurable improvements in performance.

6 Expected Results

We expect the learned configuration to outperform default HNSW settings in at least one of the key metrics while maintaining acceptable trade-offs between accuracy, speed, and memory.

7 Learnings and Outcomes

- Understanding of HNSW graph structures
- Experience with Bayesian optimization
- Insights into performance trade-offs in ANN systems

8 Conclusion

This project aims to bridge the gap between heuristic-based and data-driven graph construction in ANN systems. By learning optimal configurations, we aim to build more efficient and adaptive vector search systems.

References

[1] Y. A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence.*

[2] J. Johnson, M. Douze, and H. Jégou. 2019. Billion-scale similarity search with FAISS. *IEEE Transactions on Big Data.*

[3] Pinecone. HNSW Explained. https://www.pinecone.io/learn/series/faiss/hnsw/